INTE 201: PROGRAMMING 3 (Structured Programming)

Name: Gueta, Ethan Ulrick D.
Instructor: Sir Victor James Ordoñez
Year and Section: DIT 2-3
Date Passed: October 25, 2026

Activities – Week 1 – Data Types & Variables
Primitive Types • Reference Types • Variables • Declaration • Initialization • Type Casting

BASIC

Personal Profile Printer

Declare and initialize variables describing yourself, using a mix of primitive and reference types, then print a simple profile.

Requirements:

- Declare at least 4 different primitive-type variables (e.g., int, double, char, boolean) representing age, height, initial, and student status.
- Declare at least 1 reference-type variable (a String) for your full name.
- Initialize every variable at the moment of declaration.
- Print all values on separate, clearly labeled lines.

Program Link: https://www.jdoodle.com/ia/juvg-kjy5-qjtf

Source Code:

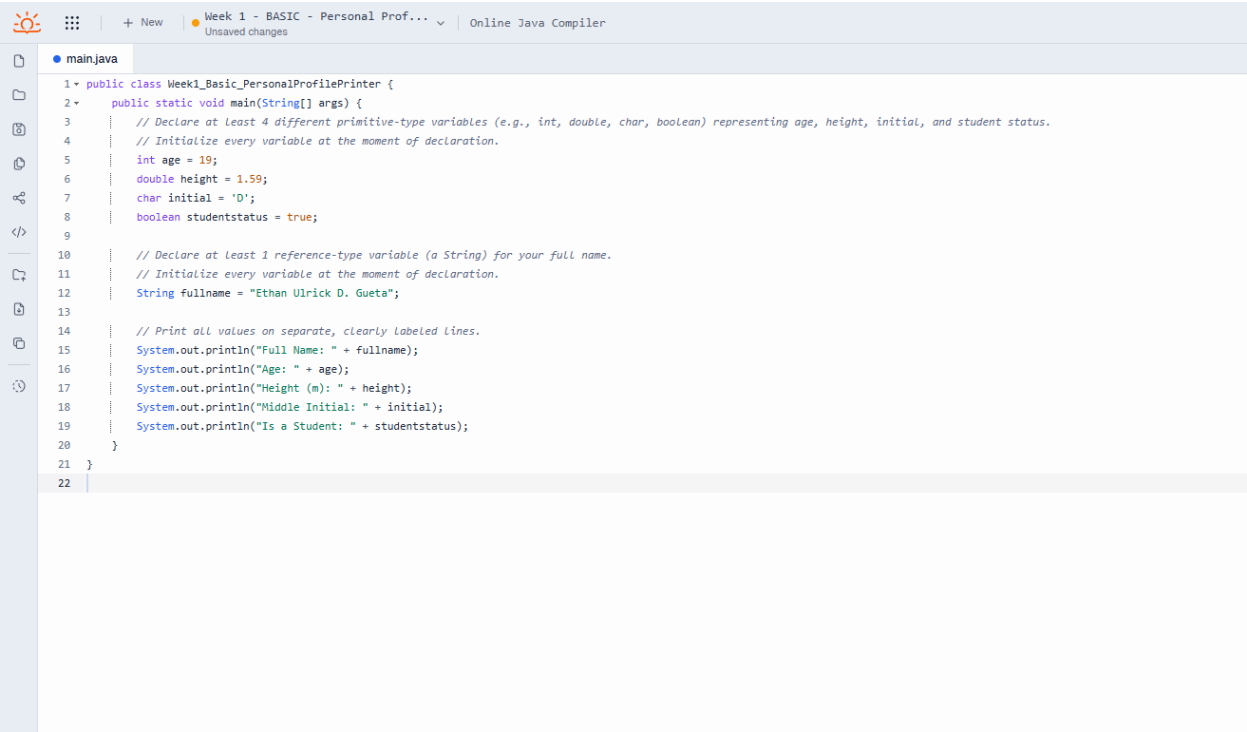
```
public class Week1_Basic_PersonalProfilePrinter {
  public static void main(String[] args) {
    // Declare at least 4 different primitive-type variables (e.g., int, double, char, boolean) representing age, height, initial, and student status.
    // Initialize every variable at the moment of declaration.
    int age = 19;
    double height = 1.59;
    char initial = 'D';
    boolean studentstatus = true;

    // Declare at least 1 reference-type variable (a String) for your full name.
    // Initialize every variable at the moment of declaration.
    String fullname = "Ethan Ulrick D. Gueta";

    // Print all values on separate, clearly labeled lines.
    System.out.println("Full Name: " + fullname);
    System.out.println("Age: " + age);
    System.out.println("Height (m): " + height);
    System.out.println("Middle Initial: " + initial);
    System.out.println("Is a Student: " + studentstatus);
  }
}
```

Screenshots:

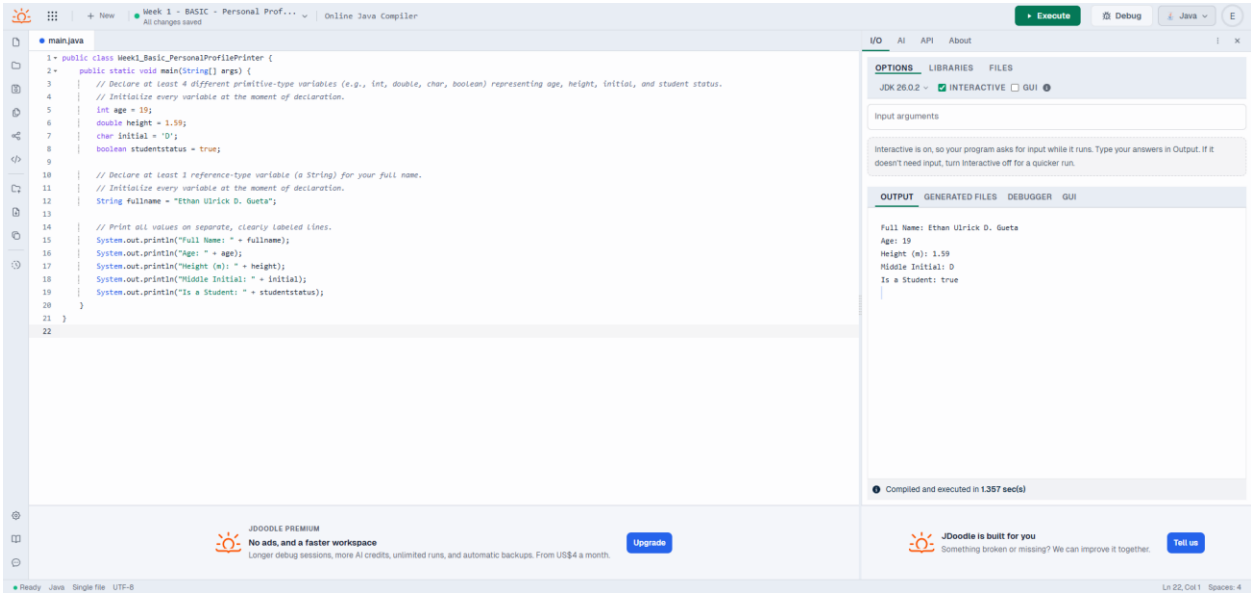
Input (Code):



Output:



Final Screenshot:



Compiler Used:
https://www.jdoodle.com/online-java-compiler

INTERMEDIATE

Type-Safe Shopping Calculator

Model a small purchase using correctly chosen types, then compute and print the total cost.

Requirements:

- Declare quantity (int), unitPrice (double), and storeName (String) with sensible initial values.
- Compute totalCost using the correct arithmetic, storing the result in a properly typed variable.
- Print an itemized summary including the store name, quantity, unit price, and total.
- In a comment, explain why totalCost should NOT be declared as an int.

Program Link: https://www.jdoodle.com/ia/8hw7-y5wb-kqqx

Source Code:

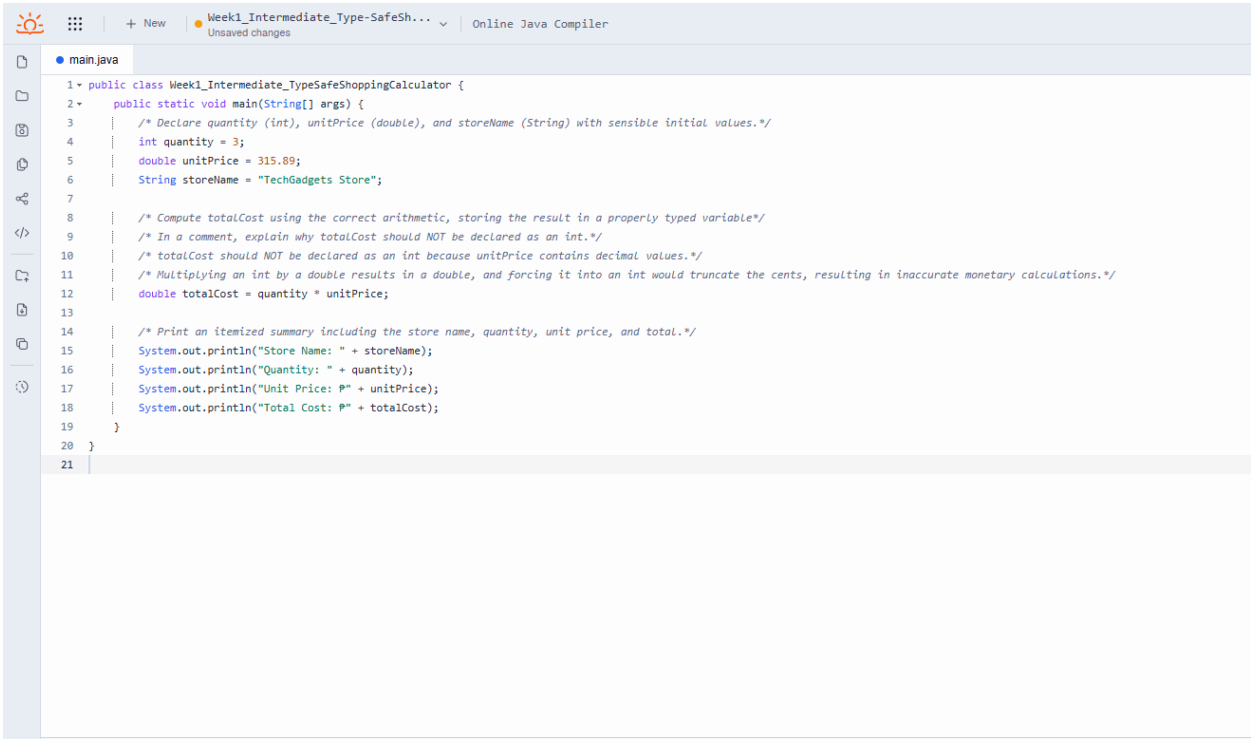
```
public class Week1_Intermediate_TypeSafeShoppingCalculator {
  public static void main(String[] args) {
    /* Declare quantity (int), unitPrice (double), and storeName (String) with sensible initial values.*/
    int quantity = 3;
    double unitPrice = 315.89;
    String storeName = "TechGadgets Store";

    /* Compute totalCost using the correct arithmetic, storing the result in a properly typed variable*/
    /* In a comment, explain why totalCost should NOT be declared as an int.*/
    /* totalCost should NOT be declared as an int because unitPrice contains decimal values.*/
    /* Multiplying an int by a double results in a double, and forcing it into an int would truncate the cents, resulting in inaccurate monetary calculations.*/
    double totalCost = quantity * unitPrice;

    /* Print an itemized summary including the store name, quantity, unit price, and total.*/
    System.out.println("Store Name: " + storeName);
    System.out.println("Quantity: " + quantity);
    System.out.println("Unit Price: ₱" + unitPrice);
    System.out.println("Total Cost: ₱" + totalCost);
  }
}
```

Screenshots:

Input (Code):



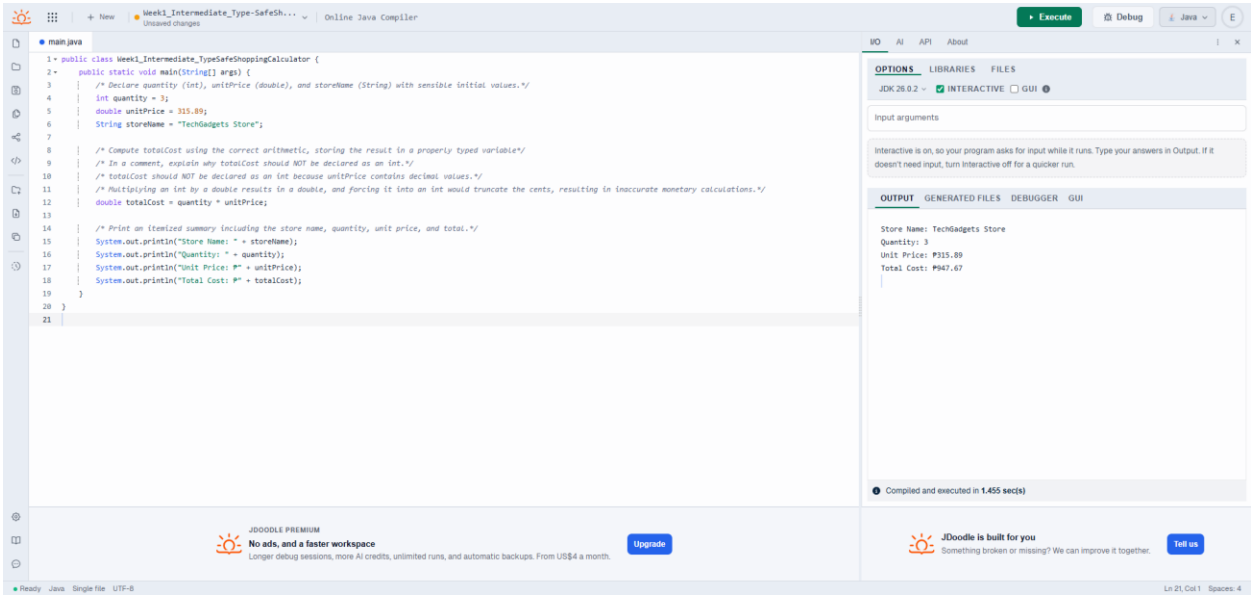
Output:

```
OUTPUT  GENERATED FILES  DEBUGGER  GUI

Store Name: TechGadgets Store
Quantity: 3
Unit Price: ₱315.89
Total Cost: ₱947.67

Compiled and executed in 1.455 sec(s)
```

Final Screenshot:



Compiler Used:

https://www.jdoodle.com/online-java-compiler

ADVANCED

Unit Casting Converter

Build a small converter that demonstrates both implicit widening and explicit narrowing casts correctly.

Requirements:

- Read or declare a double value representing centimeters.
- Use explicit casting to compute and print the equivalent WHOLE meters, plus the remaining centimeters.
- Separately, demonstrate one implicit widening example (int to double) with a comment explaining why no cast is needed.
- Demonstrate one explicit narrowing example (double to int) with a comment explaining exactly what data is lost.

Program Link: https://www.jdoodle.com/ia/y4tr-v7fp-mu77

Source Code:

```
public class Week1_Advanced_UnitCastingConverter {
  public static void main(String[] args) {
    /* Read or declare a double value representing centimeters */
    double totalCentimeters = 294.6;

    /* •  Use explicit casting to compute and print the equivalent WHOLE meters, plus the remaining centimeters. */
    int wholeMeters = (int) (totalCentimeters / 100);
    double remainingCentimeters = totalCentimeters % 100;

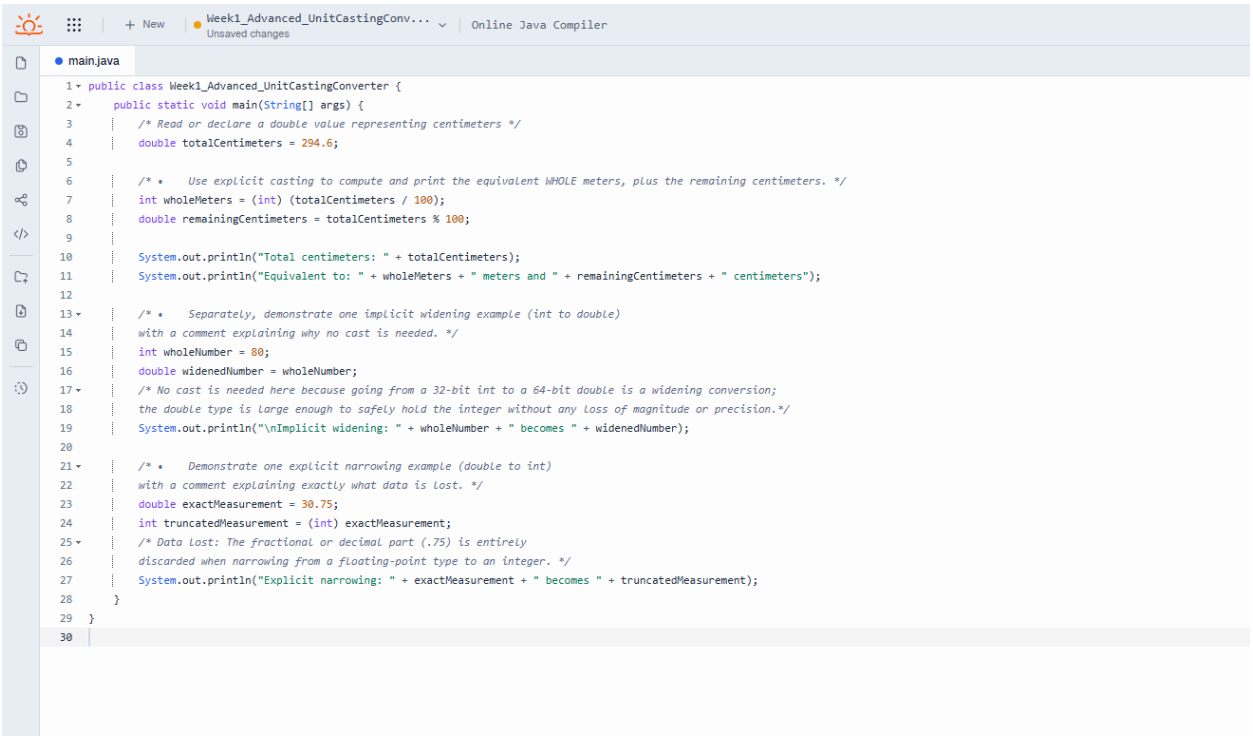
    System.out.println("Total centimeters: " + totalCentimeters);
    System.out.println("Equivalent to: " + wholeMeters + " meters and " + remainingCentimeters + " centimeters");

    /* •  Separately, demonstrate one implicit widening example (int to double)
    with a comment explaining why no cast is needed. */
    int wholeNumber = 80;
    double widenedNumber = wholeNumber;
    /* No cast is needed here because going from a 32-bit int to a 64-bit double is a widening conversion;
    the double type is large enough to safely hold the integer without any loss of magnitude or precision.*/
    System.out.println("\nImplicit widening: " + wholeNumber + " becomes " + widenedNumber);

    /* •  Demonstrate one explicit narrowing example (double to int)
    with a comment explaining exactly what data is lost. */
    double exactMeasurement = 30.75;
    int truncatedMeasurement = (int) exactMeasurement;
    /* Data lost: The fractional or decimal part (.75) is entirely
    discarded when narrowing from a floating-point type to an integer. */
    System.out.println("Explicit narrowing: " + exactMeasurement + " becomes " + truncatedMeasurement);
  }
}
```

Screenshots:

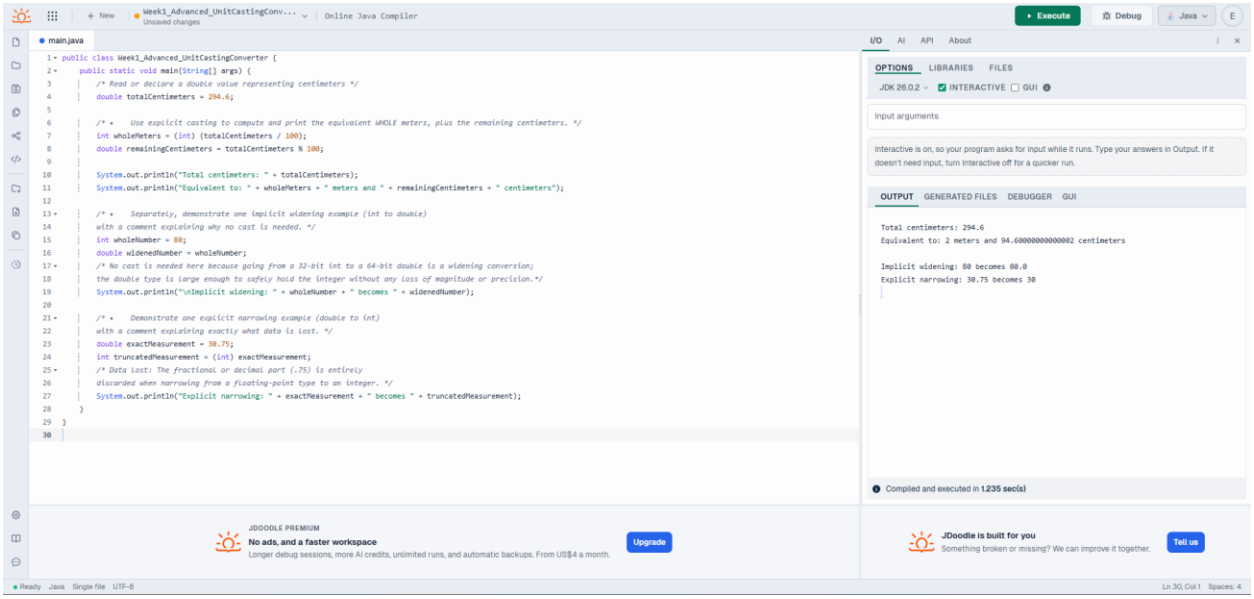
Input (Code):



Output:



Final Screenshot:



Compiler Used:

https://www.jdoodle.com/online-java-compiler

EXPERT

Overflow & Reference Investigator

Combine primitive overflow behavior with reference-type sharing behavior in one investigative program.

Requirements:

- Demonstrate a byte or short variable overflowing (wrapping around) after an increment, printing the unexpected result.
- Create two String variables using new String("...") with the same text, and one using a literal; compare all pairs with == and.equals(), printing every result.
- In comments, explain WHY each == comparison produced the result it did.
- Demonstrate at least one array reference shared between two variables, showing that modifying through one is visible through the other.

Program Link: https://www.jdoodle.com/ia/9jes-vcte-zabb

Source Code:

```
import java.util.Arrays;

public class Week1_Expert_OverflowReferenceInvestigator {
  public static void main(String[] args) {
    /* Demonstrate a byte variable overflowing */
    byte maxByte = 127;
    maxByte++;
    System.out.println("Byte Overflow: " + maxByte); // Outputs: -128

    /* Create String variables using new String */
    String a = new String("test");
    String b = new String("test");

    /* WHY: 'a == b' evaluates to false because they are created with the 'new' keyword, */
    /* which allocates two distinct objects in memory with different reference addresses. */
    System.out.println("a == b: " + (a == b)); /* Outputs: false */
    System.out.println("a equals b: " + a.equals(b)); /* Outputs: true */

    /* Demonstrate array reference shared between two variables */
    int[] first = {10, 20, 30};
    int[] second = first; /* Shared reference */

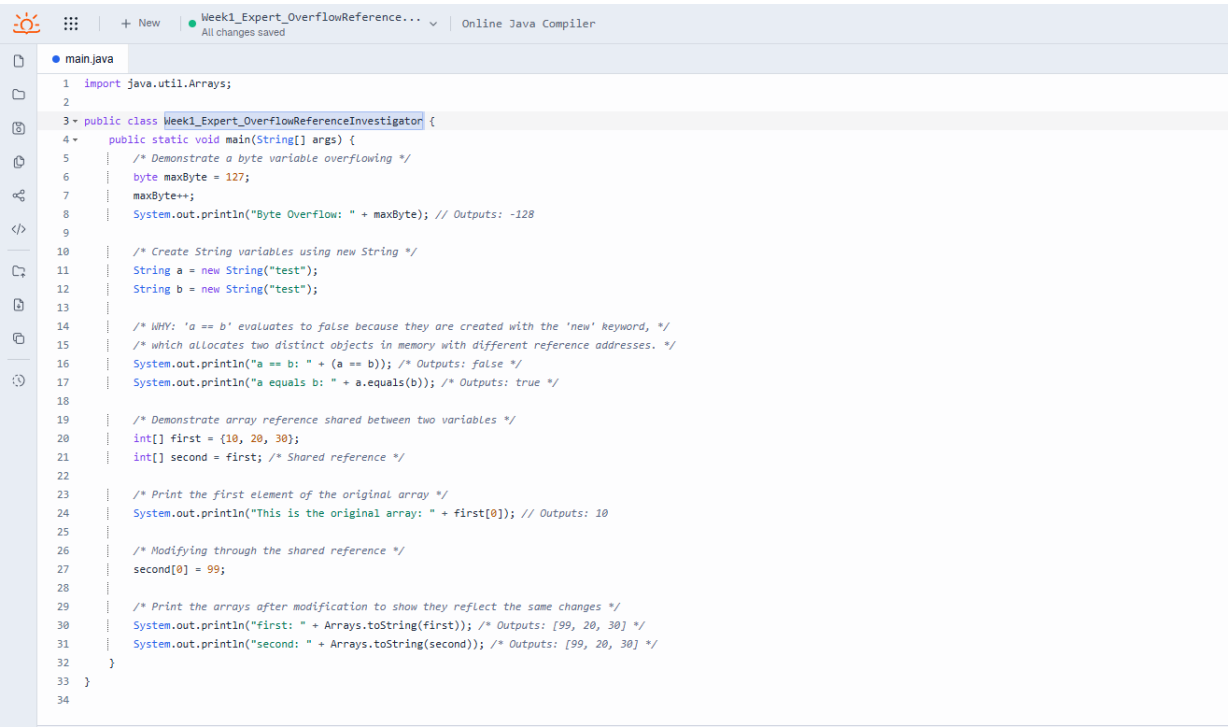
    /* Print the first element of the original array */
    System.out.println("This is the original array: " + first[0]); // Outputs: 10

    /* Modifying through the shared reference */
    second[0] = 99;

    /* Print the arrays after modification to show they reflect the same changes */
    System.out.println("first: " + Arrays.toString(first)); /* Outputs: [99, 20, 30] */
    System.out.println("second: " + Arrays.toString(second)); /* Outputs: [99, 20, 30] */
  }
}
```

Screenshots:

Input (Code):



```java
import java.util.Arrays;

public class Week1_Expert_OverflowReferenceInvestigator {
    public static void main(String[] args) {
        /* Demonstrate a byte variable overflowing */
        byte maxByte = 127;
        maxByte++;
        System.out.println("Byte Overflow: " + maxByte); // Outputs: -128

        /* Create String variables using new String */
        String a = new String("test");
        String b = new String("test");

        /* WHY: 'a == b' evaluates to false because they are created with the 'new' keyword, */
        /* which allocates two distinct objects in memory with different reference addresses. */
        System.out.println("a == b: " + (a == b)); /* Outputs: false */
        System.out.println("a equals b: " + a.equals(b)); /* Outputs: true */

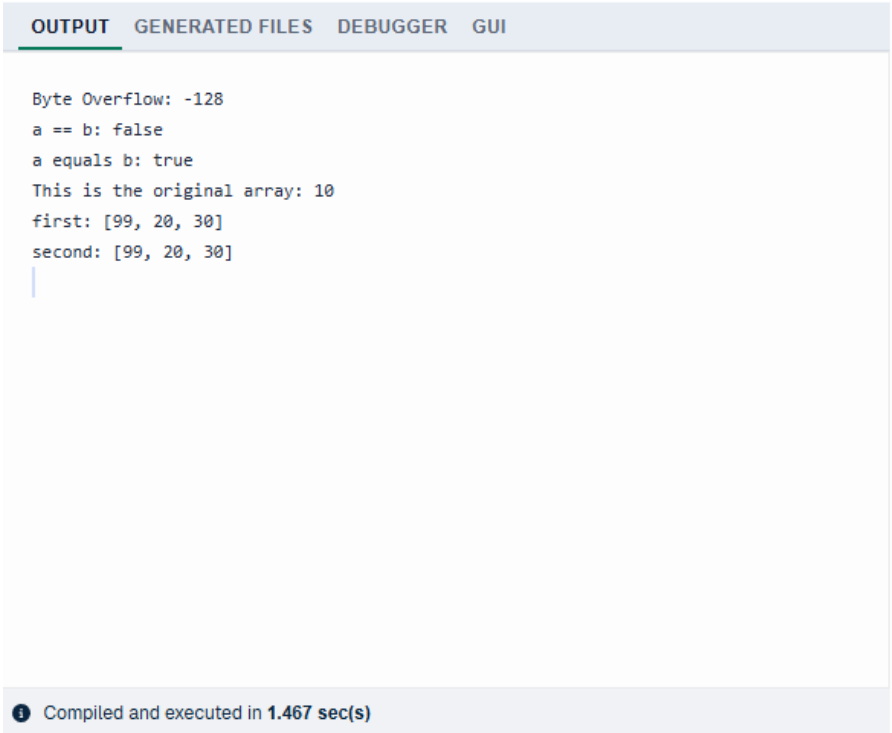
        /* Demonstrate array reference shared between two variables */
        int[] first = {10, 20, 30};
        int[] second = first; /* Shared reference */

        /* Print the first element of the original array */
        System.out.println("This is the original array: " + first[0]); // Outputs: 10

        /* Modifying through the shared reference */
        second[0] = 99;

        /* Print the arrays after modification to show they reflect the same changes */
        System.out.println("first: " + Arrays.toString(first)); /* Outputs: [99, 20, 30] */
        System.out.println("second: " + Arrays.toString(second)); /* Outputs: [99, 20, 30] */
    }
}
```

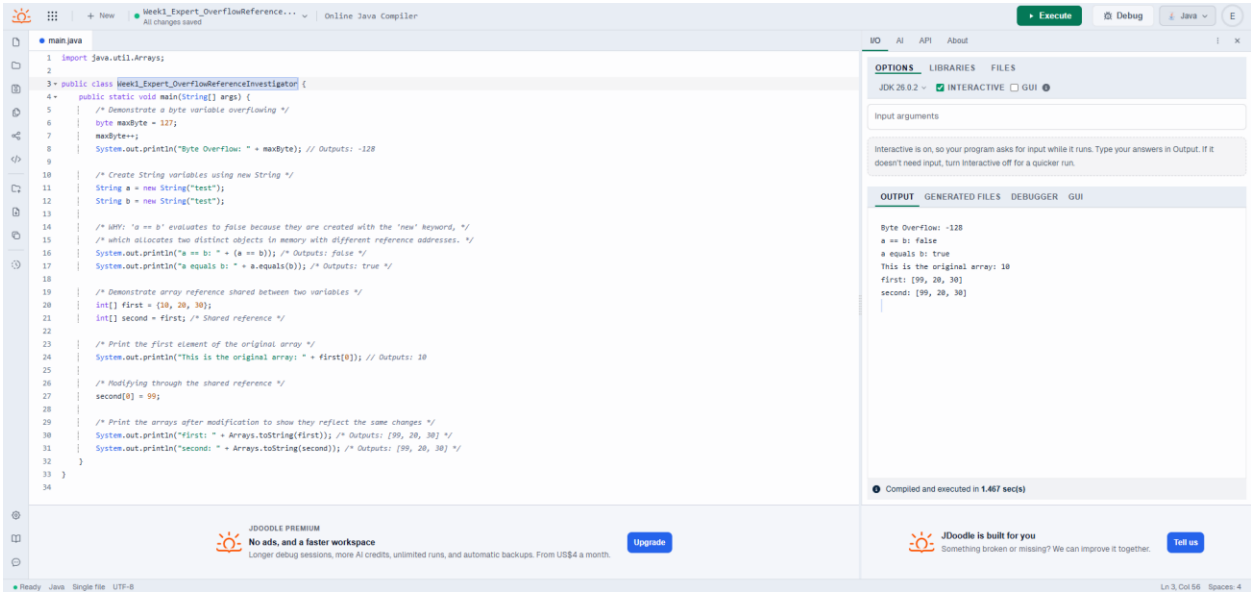
Output:



```
Byte Overflow: -128
a == b: false
a equals b: true
This is the original array: 10
first: [99, 20, 30]
second: [99, 20, 30]
```

Compiled and executed in **1.467 sec(s)**

Final Screenshot:



Compiler Used:

https://www.jdoodle.com/online-java-compiler